

MoMo SMS Processing System REST API Report

1. Introduction to API security

The MoMo SMS processing system stores 1,691 mobile-money SMS records. Exposing that data to mobile and web clients requires an API that is easy to use but still protected. This API uses HTTP Basic Authentication: every request must carry an Authorization header with a valid username and password. Requests with missing or wrong credentials are rejected with a 401 Unauthorized response, so no transaction data can be read or changed without logging in first. The API is built with plain Python (`http.server`) and stores transactions in memory.

2. API endpoints

Method	Endpoint	Purpose	Success	Errors
GET	/transactions	List all SMS transactions	200 JSON array	401
GET	/transactions/{id}	View one transaction	200 JSON object	401, 404
POST	/transactions	Add a new transaction	201 new record	400, 401
PUT	/transactions/{id}	Update an existing record	200 updated record	400, 401, 404
DELETE	/transactions/{id}	Delete a record	200 removed record	401, 404

Example request and response (GET /transactions/1):

```
$ curl -u admin:Maureen123! http://127.0.0.1:8000/transactions/1
{"id": 1, "address": "M-Money", "body": "You have received 2000 RWF from Jane Smith ...", "amount": 2000.0}
```

Error codes: 400 = invalid JSON body, 401 = missing/wrong credentials, 404 = unknown id or route.

3. DSA comparison linear search vs dictionary lookup

Both methods find a transaction by id over the parsed data set. The test searched 20 random record ids, repeated over several runs, and averaged the totals:

Method	Total time (average)	Complexity
Linear search (scan the list)	0.000646 s	O(N)
Dictionary lookup (by_id[id])	0.000003 s	O(1)

The dictionary lookup is about 195x faster in this test (dataset size: 1,691 records).

Why is dictionary lookup faster? A dictionary stores each value at a slot computed from a hash of its key, so a lookup jumps straight to the value in O(1) time. Linear search compares every element from the start, so in the worst case it scans all N records O(N) time. The difference grows as more records are added.

Suggested alternative: sort the records by id and use binary search, which finds an id in O(log N) comparisons. It is still slower than a dictionary for a single lookup but uses a plain list and keeps records ordered; a good choice when memory is limited or the list is static.

4. Reflection limitations of Basic Auth

Basic Authentication sends a username and password as Base64 text in every request header. Base64 is encoding, not encryption anyone who intercepts the traffic can decode the password, unless the whole exchange runs over HTTPS. It also has no expiry: the credential stays valid until changed, and it cannot be scoped, so one leaked password exposes every endpoint.

Stronger alternatives: JWT (JSON Web Tokens) a signed, expiring token issued after login, so passwords are not repeated on every request and tokens can be revoked; and OAuth2 delegated access with scopes and refresh tokens, the standard for letting apps and third parties connect safely. For production, the API should move off `http.server` and Basic Auth to a framework with token-based auth behind HTTPS.